Refatoração do Backend para Vertical Slice, Clean Architecture e SOLID

Plataforma de Telemetria e Gestão de Entregas (PJBL)

Disciplina: Arquitetura de Soluções / Computação em Nuvem - PUC

Grupo: Fernando Padilha, Hector Saldivar, Theo Otto, Vinicius Veiga

Repositório GitHub

https://github.com/ViniciusVeiga2002/pjbl_arq_cloud

Branch desta entrega

`refactor/backend-vertical-slice-clean-architecture`

Atividade desenvolvida com apoio de IA generativa (Claude Code / Anthropic), conforme registrado na seção 4 deste documento e em Prompt.md, na raiz do repositório - uso permitido e exigido pela disciplina (restrição OC02 do documento Arc42 do projeto).

1. Autoria e contribuição de cada aluno

Esta atividade foi conduzida em grupo, com apoio de IA generativa para a geração e revisão do código. Para fins de registro acadêmico, a tarefa foi dividida nas quatro etapas abaixo - uma por camada da arquitetura resultante - e atribuída a cada integrante do grupo, independente do caráter coletivo da atividade.

Fernando Padilha

Etapa: 1/4 - Camada de Domínio (Domain)

Levantamento das regras de negócio que estavam espalhadas pelas 4 Azure Functions originais e modelagem da camada de domínio em api/src/dominio/: a entidade Pedido (criação, geração do id público, regra de atualização parcial), o Value Object StatusPedido (conjunto fechado de status) e os erros de domínio ErroDeValidacao e PedidoNaoEncontradoError. Validação de que nenhuma regra de negócio ficou fora do domínio.

Hector Saldivar

Etapa: 2/4 - Vertical Slices e Casos de Uso (Application)

Definição da estrutura de pastas por fatia vertical (api/src/features/pedidos/cadastrar-pedido, pesquisar-pedidos, alterar-pedido, excluir-pedido) e implementação, em cada slice, do Comando/Filtro de entrada, do Caso de Uso e da Porta (interface) que ele exige do repositório - aplicando Dependency Inversion (o caso de uso depende só da porta) e Interface Segregation (uma porta por slice, com um único método cada).

Theo Otto

Etapa: 3/4 - Infraestrutura e Composição (Infrastructure)

Adaptação do acesso ao MongoDB Atlas às quatro portas definidas pelos casos de uso, em RepositorioDePedidosMongo.js (fábricas de inserção, consulta, atualização e exclusão) e ConexaoMongoDB.js (conexão cacheada, mesma estratégia do antigo mongoClient.js). Criação dos helpers HTTP RespostaHttp e mapearErroParaHttp, e do Composition Root (ContainerDeDependencias.js), único módulo que injeta o repositório concreto nos casos de uso.

Vinicius Veiga

Etapa: 4/4 - Testes, Diagramas e Entrega

Condução dos prompts à IA generativa para a refatoração; escrita dos 19 testes de unidade (node --test) com repositórios falsos em memória, validando que o contrato HTTP com o frontend não mudou; geração dos diagramas de classes e componentes do backend (PlantUML, markdown + imagem); criação da branch refactor/backend-vertical-slice-clean-architecture; consolidação desta documentação de entrega e do registro de prompts em Prompt.md.

2. Repositório GitHub e branch desta entrega

Repositório do projeto:

https://github.com/ViniciusVeiga2002/pjbl_arq_cloud

Esta refatoração foi feita em uma branch dedicada, separada da main:

`refactor/backend-vertical-slice-clean-architecture`

https://github.com/ViniciusVeiga2002/pjbl_arq_cloud/tree/refactor/backend-vertical-slice-clean-architecture

- Arquivos alterados: api/package.json (glob de entrada das Functions + script de teste).

- Arquivos removidos: api/src/functions/*.js (4 arquivos) e api/src/lib/mongoClient.js.
- Pastas novas: api/src/dominio/, api/src/features/pedidos/, api/src/infraestrutura/, api/src/composicao/ e api/tests/.
- Documentação nova: docs/backend-arquitetura/ (diagramas + markdown) e este PDF.

3. O que foi aplicado no backend

Vertical Slice Architecture

O CRUD de pedidos deixou de ser 4 arquivos soltos em api/src/functions/ e passou a ser 4 fatias verticais em api/src/features/pedidos/<slice>/ - cadastrar-pedido, pesquisar-pedidos, alterar-pedido e excluir-pedido. Cada fatia contém, de ponta a ponta, tudo que aquele caso de uso precisa: a Function HTTP, o Comando/Filtro de entrada, o Caso de Uso e a Porta (interface) para persistência.

Clean Architecture

Dentro de cada fatia (e no projeto como um todo), a Regra de Dependência é respeitada: Domínio (api/src/dominio/) não depende de nada; Aplicação (os Casos de Uso) depende só do Domínio e das Portas (abstrações); Infraestrutura (api/src/infraestrutura/, o MongoDB) implementa as Portas, nunca o contrário; e a Composição (api/src/composicao/ContainerDeDependencias.js) é o único ponto do sistema que conhece a implementação concreta e a injeta nos casos de uso.

SOLID

- S - Single Responsibility: cada classe (entidade, comando, caso de uso, adaptador HTTP) muda por um único motivo.
- O - Open/Closed: mapearErroParaHttp.js traduz novos erros de domínio sem exigir alteração nas 4 Functions.
- L - Liskov Substitution: qualquer objeto que satisfaça uma Porta substitui o repositório Mongo sem quebrar o caso de uso - é o que os 19 testes fazem.
- I - Interface Segregation: 4 portas pequenas, uma por slice, cada uma com um único método (inserir / buscarComFiltro / atualizarPorId / excluirPorId).
- D - Dependency Inversion: os Casos de Uso dependem só das Portas; o Composition Root é quem injeta o MongoDB.

Compatibilidade: nenhuma rota, método HTTP, status code ou formato de resposta mudou - o frontend React continua funcionando sem alterações. A única mudança de comportamento deliberada foi passar a validar o campo status contra o mesmo conjunto fechado já usado pela tela (Aguardando coleta / Em rota / Entregue / Atrasado), antes garantido só na UI.

4. Prompts utilizados para modificar a aplicação

Registro completo e atualizado em Prompt.md, na raiz do repositório (convenção adotada desde o início do projeto, exigida pela restrição OC02 do documento Arc42). Abaixo, o prompt que originou esta refatoração e o prompt de ajuste de prioridade pedidos nesta sessão - reproduzidos na íntegra, como digitados pelo autor.

Ferramenta: Claude Code (Anthropic)

Prompt principal

```
aplique o VERTICAL SLICE e CLEAN ARCHITECTURE e SOLID no backend da
aplicação.

Em documento PDF entregar:

No arquivo deve constar o nome dos ALUNOS que auxiliou na tarefa,
independente de ser atividade em grupo. (Descreva o que cada aluno
realizou). (tem o Fernando Padilha, Hector Saldivar, Theo Otto e
Vinicius veiga, coloque uma etapa para cada)

Informar o GITHUB do projeto em uma nova branch.

Informar todos os prompts utilizados para modificar a aplicação.

Entregar diagrama de classes e componentes do BACKEND da aplicação em
VERTICAL SLICE e CLEAN ARCHITECTURE e SOLID (gerar em markdown e
imagem).
```

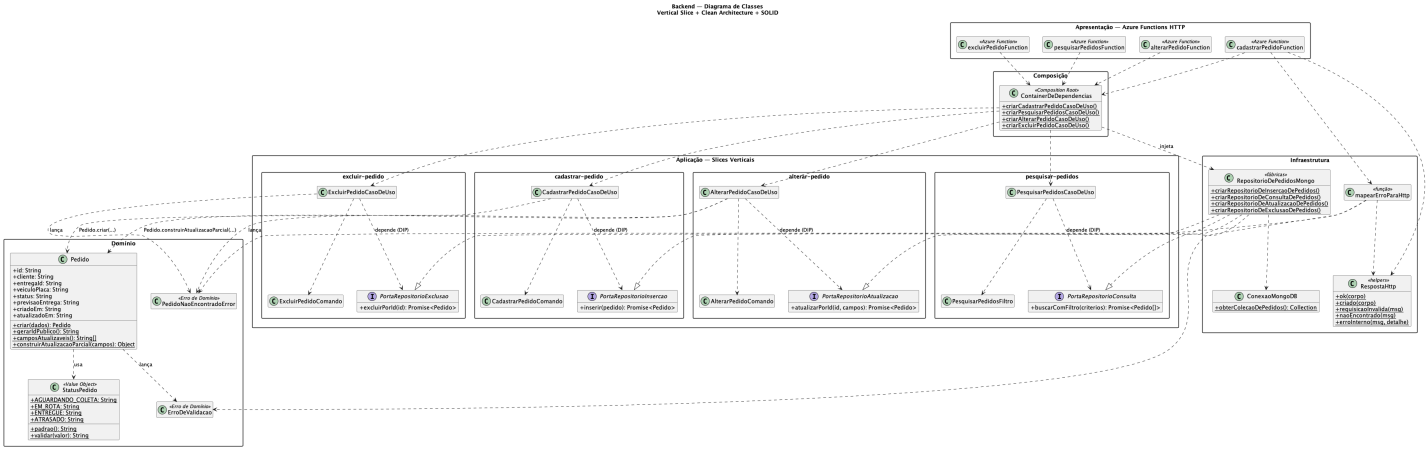
Prompt de ajuste de prioridade

```
me de primeiro a documentaçao PDF, depois faça as correcoes no codigo.
Preciso entregar o pdf o quanto antes
```

Histórico completo de prompts do projeto (frontend inicial, dados aleatórios/Apidog, CRUD com MongoDB e esta refatoração), com o que foi alterado em cada ajuste, em Prompt.md.

5. Diagrama de Classes do Backend

Fonte PlantUML e versão Markdown em docs/backend-arquitetura/arquitetura-backend.md; arquivo .puml completo em docs/backend-arquitetura/diagramas/backend-classes.puml.



6. Diagrama de Componentes do Backend

Mostra a Regra de Dependência da Clean Architecture entre as camadas e como cada fatia vertical (slice) atravessa Apresentação, Aplicação e Infraestrutura. Fonte PlantUML e versão Markdown no mesmo arquivo da seção anterior; .puml completo em docs/backend-arquitetura/diagramas/backend-componentes.puml.

